

```
In [1]:
```

```
import pandas as pd
```

```
In [2]:
```

```
df = pd.read_csv(r'Sample.csv')
```

```
df
```

	Row	Order	Order	Ship	Ship	Customer	Customer		Segment	Country/Region		Postal	Re
	ID	ID	Date	Date	Mode	ID	Name				City...	Code	
0	1	US- 2019- 103800	03- 01- 2019	07- 01- 2019	Standard Class	DP-13000	Darren Powers		Consumer	United States	Houston	...77095 Ce	
1	2	US- 2019- 112326	04- 01- 2019	08- 01- 2019	Standard Class	PO-19195	Phillina Ober	Home Office		United States	Naperville	...60540Ce	
2	3	US- 2019- 112326	04- 01- 2019	08- 01- 2019	Standard Class	PO-19195	Phillina Ober	Home Office		United States	Naperville	...60540Ce	
3	4	US- 2019- 112326	04- 01- 2019	08- 01- 2019	Standard Class	PO-19195	Phillina Ober	Home Office		United States	Naperville	...60540Ce	
4	5	US- 2019- 141817	05- 01- 2019	12- 01- 2019	Standard Class	MB-18085	Mick Brown	Consumer	United States	Philadelphia	...19143 Ea		
...	...	...	...	...	...	...	...	...	...	...	...		...
10189	10190	US- 2022- 143259	30- 12- 2022	03- 01- 2023	Standard Class	PO-18865	Patrick O'Donnell	Consumer	United States	New York City	...10009 Ea		
10190	10191	US- 2022- 115427	30- 12- 2022	03- 01- 2023	Standard Class	EB-13975	Erica Bern	Corporate	United States	Fairfield	...94533We		
10191	10192	US- 2022- 156720	30- 12- 2022	03- 01- 2023	Standard Class	JM-15580	Jill Matthias	Consumer	United States	Loveland	...80538We		
10192	10193	US- 2022- 143259	30- 12- 2022	03- 01- 2023	Standard Class	PO-18865	Patrick O'Donnell	Consumer	United States	New York City	...10009 Ea		
10193	10194	CA- 2022- 143500	30- 12- 2022	03- 01- 2023	Standard Class	HO-15230	Harry Olson	Consumer	Canada	Charlottetown...	C0A	Ea	
10194 rows × 21 columns													

	Row ID	Order ID	Order Date	Ship Date	Ship Mode \
0	1	US-2019-103800	03-01-2019	07-01-2019	Standard Class
1	2	US-2019-112326	04-01-2019	08-01-2019	Standard Class
2	3	US-2019-112326	04-01-2019	08-01-2019	Standard Class
3	4	US-2019-112326	04-01-2019	08-01-2019	Standard Class
4	5	US-2019-141817	05-01-2019	12-01-2019	Standard Class
...	...	...	...	...	...
10189	10190	US-2022-143259	30-12-2022	03-01-2023	Standard Class
10190	10191	US-2022-115427	30-12-2022	03-01-2023	Standard Class
10191	10192	US-2022-156720	30-12-2022	03-01-2023	Standard Class
10192	10193	US-2022-143259	30-12-2022	03-01-2023	Standard Class
10193	10194	CA-2022-143500	30-12-2022	03-01-2023	Standard Class

	Customer ID	Customer Name	Segment	Country/Region \
0	DP-13000	Darren Powers	Consumer	United States
1	PO-19195	Phillina Ober	Home Office	United States
2	PO-19195	Phillina Ober	Home Office	United States
3	PO-19195	Phillina Ober	Home Office	United States
4	MB-18085	Mick Brown	Consumer	United States
...	...	...	...	...
10189	PO-18865	Patrick O'Donnell	Consumer	United States
10190	EB-13975	Erica Bern	Corporate	United States
10191	JM-15580	Jill Matthias	Consumer	United States
10192	PO-18865	Patrick O'Donnell	Consumer	United States
10193	HO-15230	Harry Olson	Consumer	Canada

	City	Postal Code	Region	Product ID \
0	Houston	77095	Central	OFF-PA-10000174
1	Naperville	60540	Central	OFF-BI-10004094
2	Naperville	60540	Central	OFF-LA-10003223
3	Naperville	60540	Central	OFF-ST-10002743
4	Philadelphia	19143	East	OFF-AR-10003478
...	...	...	...	...
10189	New York City	10009	East	OFF-BI-10003684
10190	Fairfield	94533	West	OFF-BI-10004632
10191	Loveland	80538	West	OFF-FA-10003472
10192	New York City	10009	East	TEC-PH-10004774
10193	Charlottetown	C0A	East	OFF-BI-10004040

	Category	Sub-Category \
0	Office Supplies	Paper
1	Office Supplies	Binders
2	Office Supplies	Labels
3	Office Supplies	Storage
4	Office Supplies	Art
...	...	...
10189	Office Supplies	Binders
10190	Office Supplies	Binders
10191	Office Supplies	Fasteners
10192	Technology	Phones
10193	Office Supplies	Binders

	Product Name	Sales	Quantity \
0	Message Book, Wirebound, Four 5 1/2" X 4" Form...	16.448	2
1	GBC Standard Plastic Binding Systems Combs	3.540	2
2	Avery 508	11.784	3
3	SAFCO Boltless Steel Shelving	272.736	3
4	Avery Hi-Liter EverBold Pen Style Fluorescent ...	19.536	3
...	...	...	...
10189	Wilson Jones Legal Size Ring Binders	52.776	3
10190	GBC Binding covers	20.720	2
10191	Bagged Rubber Bands	3.024	3
10192	Gear Head AU3700S Headset	90.930	7
10193	Wilson Jones Impact Binders	3.024	3

	Discount	Profit
0	0.2	5.5512
1	0.8	-5.4870
2	0.2	4.2717

```
3          0.2 -64.7748
4          0.2  4.8840
...        ...      ...
10189       0.2 19.7910
10190       0.2  6.4750
10191       0.2 -0.6048
10192       0.0  2.7279
10193       0.2 -0.6048
```

```
[10194 rows x 21 columns]
```

```
In [8]:
```

```
# 1. Write a function to calculate 7-day rolling average of Sales.
```

```
def rolling_avg_sales(df):
    df = df.sort_values('Order Date')
    df['7_day_avg_sales'] = df['Sales'].rolling(7).mean()
    return df

print(rolling_avg_sales(df))
```

stdout:

	Row ID	Order ID	Order Date	Ship Date	Ship Mode \
0	1	US-2019-103800	2019-01-03	07-01-2019	Standard Class
1	2	US-2019-112326	2019-01-04	08-01-2019	Standard Class
2	3	US-2019-112326	2019-01-04	08-01-2019	Standard Class
3	4	US-2019-112326	2019-01-04	08-01-2019	Standard Class
4	5	US-2019-141817	2019-01-05	12-01-2019	Standard Class
...	...	...	...	...	...
10188	10189	US-2022-115427	2022-12-30	03-01-2023	Standard Class
10189	10190	US-2022-143259	2022-12-30	03-01-2023	Standard Class
10190	10191	US-2022-115427	2022-12-30	03-01-2023	Standard Class
10191	10192	US-2022-156720	2022-12-30	03-01-2023	Standard Class
10193	10194	CA-2022-143500	2022-12-30	03-01-2023	Standard Class

	Customer ID	Customer Name	Segment	Country/Region \
0	DP-13000	Darren Powers	Consumer	United States
1	PO-19195	Phillina Ober	Home Office	United States
2	PO-19195	Phillina Ober	Home Office	United States
3	PO-19195	Phillina Ober	Home Office	United States
4	MB-18085	Mick Brown	Consumer	United States
...	...	...	...	...
10188	EB-13975	Erica Bern	Corporate	United States
10189	PO-18865	Patrick O'Donnell	Consumer	United States
10190	EB-13975	Erica Bern	Corporate	United States
10191	JM-15580	Jill Matthias	Consumer	United States
10193	HO-15230	Harry Olson	Consumer	Canada

	City	...	Region	Product ID	Category \
0	Houston	...	Central	OFF-PA-10000174	Office Supplies
1	Naperville	...	Central	OFF-BI-10004094	Office Supplies
2	Naperville	...	Central	OFF-LA-10003223	Office Supplies
3	Naperville	...	Central	OFF-ST-10002743	Office Supplies
4	Philadelphia	...	East	OFF-AR-10003478	Office Supplies
...	...	...	...	...	...
10188	Fairfield	...	West	OFF-BI-10002103	Office Supplies
10189	New York City	...	East	OFF-BI-10003684	Office Supplies
10190	Fairfield	...	West	OFF-BI-10004632	Office Supplies
10191	Loveland	...	West	OFF-FA-10003472	Office Supplies
10193	Charlottetown	...	East	OFF-BI-10004040	Office Supplies

	Sub-Category	Product Name \
0	Paper	Message Book, Wirebound, Four 5 1/2" X 4" Form...
1	Binders	GBC Standard Plastic Binding Systems Combs
2	Labels	Avery 508
3	Storage	SAFCO Boltless Steel Shelving
4	Art	Avery Hi-Liter EverBold Pen Style Fluorescent ...
...	...	...
10188	Binders	Cardinal Slant-D Ring Binder, Heavy Gauge Vinyl
10189	Binders	Wilson Jones Legal Size Ring Binders
10190	Binders	GBC Binding covers
10191	Fasteners	Bagged Rubber Bands
10193	Binders	Wilson Jones Impact Binders

	Sales Quantity	Discount	Profit	7_day_avg_sales	
0	16.448	2	0.2	5.5512	NaN
1	3.540	2	0.8	-5.4870	NaN
2	11.784	3	0.2	4.2717	NaN
3	272.736	3	0.2	-64.7748	NaN
4	19.536	3	0.2	4.8840	NaN
...	...	...	...	...	...
10188	13.904	2	0.2	4.5188	302.341429
10189	52.776	3	0.2	19.7910	137.332286
10190	20.720	2	0.2	6.4750	103.328000
10191	3.024	3	0.2	-0.6048	101.970000
10193	3.024	3	0.2	-0.6048	89.412000

[10194 rows x 22 columns]

```
In [11]:  
  
# 2. Write a function to perform cohort analysis based on first purchase month.  
  
def cohort(df):  
    df['Order Date'] = pd.to_datetime(df['Order Date'])  
    df['Month'] = df['Order Date'].dt.to_period('M')  
    first = df.groupby('Customer ID')['Month'].min()  
    df['Cohort'] = df['Customer ID'].map(first)  
    return df  
print(cohort(df))
```

stdout:

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	\
0	1	US-2019-103800	2019-01-03	07-01-2019	Standard Class	
1	2	US-2019-112326	2019-01-04	08-01-2019	Standard Class	
2	3	US-2019-112326	2019-01-04	08-01-2019	Standard Class	
3	4	US-2019-112326	2019-01-04	08-01-2019	Standard Class	
4	5	US-2019-141817	2019-01-05	12-01-2019	Standard Class	
...	...	...	...	...	...	
10189	10190	US-2022-143259	2022-12-30	03-01-2023	Standard Class	
10190	10191	US-2022-115427	2022-12-30	03-01-2023	Standard Class	
10191	10192	US-2022-156720	2022-12-30	03-01-2023	Standard Class	
10192	10193	US-2022-143259	2022-12-30	03-01-2023	Standard Class	
10193	10194	CA-2022-143500	2022-12-30	03-01-2023	Standard Class	

	Customer ID	Customer Name	Segment	Country/Region	\
0	DP-13000	Darren Powers	Consumer	United States	
1	PO-19195	Phillina Ober	Home Office	United States	
2	PO-19195	Phillina Ober	Home Office	United States	
3	PO-19195	Phillina Ober	Home Office	United States	
4	MB-18085	Mick Brown	Consumer	United States	
...	...	...	...	...	
10189	PO-18865	Patrick O'Donnell	Consumer	United States	
10190	EB-13975	Erica Bern	Corporate	United States	
10191	JM-15580	Jill Matthias	Consumer	United States	
10192	PO-18865	Patrick O'Donnell	Consumer	United States	
10193	HO-15230	Harry Olson	Consumer	Canada	

	City	...	Product Name	\
0	Houston	...	Message Book, Wirebound, Four 5 1/2" X 4" Form...	
1	Naperville	...	GBC Standard Plastic Binding Systems Combs	
2	Naperville	...	Avery 508	
3	Naperville	...	SAFCO Boltless Steel Shelving	
4	Philadelphia	...	Avery Hi-Liter EverBold Pen Style Fluorescent ...	
...	...	...	...	
10189	New York City	...	Wilson Jones Legal Size Ring Binders	
10190	Fairfield	...	GBC Binding covers	
10191	Loveland	...	Bagged Rubber Bands	
10192	New York City	...	Gear Head AU3700S Headset	
10193	Charlottetown	...	Wilson Jones Impact Binders	

	Sales	Quantity	Discount	Profit	7_day_avg_sales	Month	Cohort	\
0	16.448	2	0.2	5.5512	NaN	2019-01	2019-01	
1	3.540	2	0.8	-5.4870	NaN	2019-01	2019-01	
2	11.784	3	0.2	4.2717	NaN	2019-01	2019-01	
3	272.736	3	0.2	-64.7748	NaN	2019-01	2019-01	
4	19.536	3	0.2	4.8840	NaN	2019-01	2019-01	
...	...	...	...	...	...	...	...	
10189	52.776	3	0.2	19.7910	112.328000	2022-12	2019-03	
10190	20.720	2	0.2	6.4750	101.128000	2022-12	2021-01	
10191	3.024	3	0.2	-0.6048	91.780000	2022-12	2021-04	
10192	90.930	7	0.0	2.7279	101.970000	2022-12	2019-03	
10193	3.024	3	0.2	-0.6048	56.239714	2022-12	2019-12	

	Order_Month	Cohort_Month
0	2019-01	2019-01
1	2019-01	2019-01
2	2019-01	2019-01
3	2019-01	2019-01
4	2019-01	2019-01
...	...	...
10189	2022-12	2019-03
10190	2022-12	2021-01
10191	2022-12	2021-04
10192	2022-12	2019-03
10193	2022-12	2019-12

[10194 rows x 26 columns]

```

In [12]:

# 3. Write a function to calculate customer lifetime value and return top 10
# customers.

def top_customer_clv(df):
    clv = df.groupby('Customer ID')['Sales'].sum()
    return clv.sort_values(ascending=False).head(10)

print(top_customer_clv(df))

```

```

stdout:
Customer ID
SM-20320    25043.050
TC-20980    19052.218
RB-19360    15117.339
TA-21385    14595.620
AB-10105    14473.571
KL-16645    14175.229
SC-20095    14142.334
HL-15040    12873.298
SE-20110    12209.438
CC-12370    12129.072
Name: Sales, dtype: float64

```

```

In [13]:

# 4. Write a function to return total profit grouped by Region and Category sorted
# descending.

def profit_region_category(df):
    return df.groupby(['Region', 'Category'])['Profit'].sum().sort_values(ascending=False)

print(profit_region_category(df))

```

```

stdout:
Region  Category      Profit
West    Office Supplies  54070.2292
East    Technology       48441.7758
West    Technology       44412.3364
East    Office Supplies  42996.7397
Central Technology       33697.4320
South   Technology       19991.8314
        Office Supplies  19986.3928
West    Furniture       12316.2514
Central Office Supplies   8970.0817
South   Furniture        6771.2061
East    Furniture        3444.7448
Central Furniture       -2802.2067
Name: Profit, dtype: float64

```

```

In [14]:

# 5. Write a function to assign rank to orders within each Region based on
# Sales.

def rank_orders(df):
    df['Sales_Rank'] = df.groupby('Region')['Sales'].rank(ascending=False)
    return df

print(rank_orders(df).head())

```

```

stdout:
      Row ID      Order ID Order Date  Ship Date      Ship Mode Customer ID \
0         1  US-2019-103800 2019-01-03  07-01-2019  Standard Class  DP-13000
1         2  US-2019-112326 2019-01-04  08-01-2019  Standard Class  PO-19195
2         3  US-2019-112326 2019-01-04  08-01-2019  Standard Class  PO-19195
3         4  US-2019-112326 2019-01-04  08-01-2019  Standard Class  PO-19195
4         5  US-2019-141817 2019-01-05  12-01-2019  Standard Class  MB-18085

      Customer Name      Segment Country/Region      City ... Sales \
0  Darren Powers      Consumer  United States      Houston ... 16.448
1  Phillina Ober      Home Office  United States  Naperville ... 3.540
2  Phillina Ober      Home Office  United States  Naperville ... 11.784
3  Phillina Ober      Home Office  United States  Naperville ... 272.736
4    Mick Brown      Consumer  United States  Philadelphia ... 19.536

      Quantity Discount      Profit 7_day_avg_sales      Month      Cohort Order_Month \
0         2         0.2      5.5512           NaN 2019-01 2019-01      2019-01
1         2         0.8     -5.4870           NaN 2019-01 2019-01      2019-01
2         3         0.2      4.2717           NaN 2019-01 2019-01      2019-01
3         3         0.2    -64.7748           NaN 2019-01 2019-01      2019-01
4         3         0.2      4.8840           NaN 2019-01 2019-01      2019-01

      Cohort_Month  Sales_Rank
0      2019-01      1672.0
1      2019-01      2214.0
2      2019-01      1852.0
3      2019-01       471.0
4      2019-01      2156.0

[5 rows x 27 columns]

```

```

In [15]:

# 6. Write a function to create a Z-score normalized Sales column.

def zscore_sales(df):
    df['Sales_Zscore'] = (df['Sales'] - df['Sales'].mean()) / df['Sales'].std()
    return df

print(zscore_sales(df).head())

```

```

stdout:

   Row ID   Order ID Order Date   Ship Date   Ship Mode Customer ID \
0      1  US-2019-103800 2019-01-03  07-01-2019  Standard Class  DP-13000
1      2  US-2019-112326 2019-01-04  08-01-2019  Standard Class  PO-19195
2      3  US-2019-112326 2019-01-04  08-01-2019  Standard Class  PO-19195
3      4  US-2019-112326 2019-01-04  08-01-2019  Standard Class  PO-19195
4      5  US-2019-141817 2019-01-05  12-01-2019  Standard Class  MB-18085

   Customer Name   Segment Country/Region   City   ... Quantity \
0  Darren Powers   Consumer   United States   Houston   ...      2
1  Phillina Ober   Home Office   United States   Naperville   ...      2
2  Phillina Ober   Home Office   United States   Naperville   ...      3
3  Phillina Ober   Home Office   United States   Naperville   ...      3
4    Mick Brown   Consumer   United States   Philadelphia   ...      3

   Discount   Profit 7_day_avg_sales   Month   Cohort Order_Month \
0      0.2   5.5512             NaN  2019-01  2019-01   2019-01
1      0.8  -5.4870             NaN  2019-01  2019-01   2019-01
2      0.2   4.2717             NaN  2019-01  2019-01   2019-01
3      0.2 -64.7748             NaN  2019-01  2019-01   2019-01
4      0.2   4.8840             NaN  2019-01  2019-01   2019-01

   Cohort_Month   Sales_Rank   Sales_Zscore
0      2019-01      1672.0      -0.341629
1      2019-01      2214.0      -0.362451
2      2019-01      1852.0      -0.349152
3      2019-01       471.0       0.071801
4      2019-01      2156.0      -0.336647

[5 rows x 28 columns]

```

```

In [16]:

# 7. .Write a function to replace negative profits with median profit of that
# category.

def replace_negative_profit(df):
    median_profit = df.groupby('Category')['Profit'].transform('median')
    df['Profit'] = df['Profit'].mask(df['Profit'] < 0, median_profit)
    return df

print(replace_negative_profit(df).head())

```

stdout:

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	\
0	1	US-2019-103800	2019-01-03	07-01-2019	Standard Class	DP-13000	
1	2	US-2019-112326	2019-01-04	08-01-2019	Standard Class	PO-19195	
2	3	US-2019-112326	2019-01-04	08-01-2019	Standard Class	PO-19195	
3	4	US-2019-112326	2019-01-04	08-01-2019	Standard Class	PO-19195	
4	5	US-2019-141817	2019-01-05	12-01-2019	Standard Class	MB-18085	

	Customer Name	Segment	Country/Region	City	...	Quantity	\
0	Darren Powers	Consumer	United States	Houston	...	2	
1	Phillina Ober	Home Office	United States	Naperville	...	2	
2	Phillina Ober	Home Office	United States	Naperville	...	3	
3	Phillina Ober	Home Office	United States	Naperville	...	3	
4	Mick Brown	Consumer	United States	Philadelphia	...	3	

	Discount	Profit	7_day_avg_sales	Month	Cohort	Order_Month	\
0	0.2	5.5512	NaN	2019-01	2019-01	2019-01	
1	0.8	6.8922	NaN	2019-01	2019-01	2019-01	
2	0.2	4.2717	NaN	2019-01	2019-01	2019-01	
3	0.2	6.8922	NaN	2019-01	2019-01	2019-01	
4	0.2	4.8840	NaN	2019-01	2019-01	2019-01	

	Cohort_Month	Sales_Rank	Sales_Zscore
0	2019-01	1672.0	-0.341629
1	2019-01	2214.0	-0.362451
2	2019-01	1852.0	-0.349152
3	2019-01	471.0	0.071801
4	2019-01	2156.0	-0.336647

[5 rows x 28 columns]

In [17]:

```
# 8. .Write a function to create a lag feature for previous order sales per
# customer.
```

```
def lag_sales_customer(df):
    df['Order Date'] = pd.to_datetime(df['Order Date'], dayfirst=True)
    df = df.sort_values(['Customer ID', 'Order Date'])

    df['Prev_Sales'] = df.groupby('Customer ID')['Sales'].shift(1)

    df['Prev_Sales'] = df['Prev_Sales'].fillna(0)

    return df

print(lag_sales_customer(df).head(10))
```

stdout:

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	\
281	282	US-2019-128055	2019-03-31	05-04-2019	Standard Class	
287	288	US-2019-128055	2019-03-31	05-04-2019	Standard Class	
1131	1132	US-2019-138100	2019-09-15	20-09-2019	Standard Class	
1132	1133	US-2019-138100	2019-09-15	20-09-2019	Standard Class	
3395	3396	US-2020-121391	2020-10-04	07-10-2020	First Class	
4369	4370	US-2021-103982	2021-03-03	08-03-2021	Standard Class	
4372	4373	US-2021-103982	2021-03-03	08-03-2021	Standard Class	
4374	4375	US-2021-103982	2021-03-03	08-03-2021	Standard Class	
4376	4377	US-2021-103982	2021-03-03	08-03-2021	Standard Class	
8004	8005	US-2022-147039	2022-06-29	04-07-2022	Standard Class	

	Customer ID	Customer Name	Segment	Country/Region	City	...	\
281	AA-10315	Alex Avila	Consumer	United States	San Francisco	...	
287	AA-10315	Alex Avila	Consumer	United States	San Francisco	...	
1131	AA-10315	Alex Avila	Consumer	United States	New York City	...	
1132	AA-10315	Alex Avila	Consumer	United States	New York City	...	
3395	AA-10315	Alex Avila	Consumer	United States	San Francisco	...	
4369	AA-10315	Alex Avila	Consumer	United States	Round Rock	...	
4372	AA-10315	Alex Avila	Consumer	United States	Round Rock	...	
4374	AA-10315	Alex Avila	Consumer	United States	Round Rock	...	
4376	AA-10315	Alex Avila	Consumer	United States	Round Rock	...	
8004	AA-10315	Alex Avila	Consumer	United States	Minneapolis	...	

	Discount	Profit	7_day_avg_sales	Month	Cohort	Order_Month	\
281	0.0	14.8344	279.381714	2019-03	2019-03	2019-03	
287	0.2	252.5880	107.725714	2019-03	2019-03	2019-03	
1131	0.0	6.2608	525.630571	2019-09	2019-03	2019-09	
1132	0.0	7.0218	526.786571	2019-09	2019-03	2019-09	
3395	0.0	7.0096	84.653143	2020-10	2019-03	2020-10	
4369	0.2	0.7776	269.050571	2021-03	2019-03	2021-03	
4372	0.2	6.8922	751.993714	2021-03	2019-03	2021-03	
4374	0.2	5.7365	606.467143	2021-03	2019-03	2021-03	
4376	0.2	32.3982	703.750286	2021-03	2019-03	2021-03	
8004	0.0	90.7350	436.153600	2022-06	2019-03	2022-06	

	Cohort_Month	Sales_Rank	Sales_Zscore	Prev_Sales
281	2019-03	1710.0	-0.282697	0.000
287	2019-03	270.0	0.718402	52.980
1131	2019-03	2375.0	-0.344674	673.568
1132	2019-03	2344.0	-0.344061	14.560
3395	2019-03	2197.5	-0.324671	14.940
4369	2019-03	2274.5	-0.364445	26.960
4372	2019-03	8.0	5.971617	2.304
4374	2019-03	1203.0	-0.300861	3930.072
4376	2019-03	301.0	0.328679	41.720
8004	2019-03	365.0	0.217314	431.976

[10 rows x 29 columns]

In [18]:

```
# 9. Write a function to calculate cumulative sales per region over time.
```

```
def cumulative_sales_region(df):  
    df['Order Date'] = pd.to_datetime(df['Order Date'], dayfirst=True)  
  
    df = df.sort_values(['Region', 'Order Date'])  
  
    df['Cumulative_Sales'] = df.groupby('Region')['Sales'].cumsum()  
  
    return df  
  
print(cumulative_sales_region(df).head(10))
```

stdout:

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	\
0	1	US-2019-103800	2019-01-03	07-01-2019	Standard Class	DP-13000	
1	2	US-2019-112326	2019-01-04	08-01-2019	Standard Class	PO-19195	
2	3	US-2019-112326	2019-01-04	08-01-2019	Standard Class	PO-19195	
3	4	US-2019-112326	2019-01-04	08-01-2019	Standard Class	PO-19195	
14	15	US-2019-105417	2019-01-07	12-01-2019	Standard Class	VS-21820	
15	16	US-2019-105417	2019-01-07	12-01-2019	Standard Class	VS-21820	
16	17	US-2019-135405	2019-01-09	13-01-2019	Standard Class	MS-17830	
17	18	US-2019-135405	2019-01-09	13-01-2019	Standard Class	MS-17830	
44	45	US-2019-167927	2019-01-20	26-01-2019	Standard Class	XP-21865	
45	46	US-2019-167927	2019-01-20	26-01-2019	Standard Class	XP-21865	

	Customer Name	Segment	Country/Region	City	...	Discount	\
0	Darren Powers	Consumer	United States	Houston	...	0.2	
1	Phillina Ober	Home Office	United States	Naperville	...	0.8	
2	Phillina Ober	Home Office	United States	Naperville	...	0.2	
3	Phillina Ober	Home Office	United States	Naperville	...	0.2	
14	Vivek Sundaresam	Consumer	United States	Huntsville	...	0.6	
15	Vivek Sundaresam	Consumer	United States	Huntsville	...	0.8	
16	Melanie Seite	Consumer	United States	Laredo	...	0.2	
17	Melanie Seite	Consumer	United States	Laredo	...	0.2	
44	Xylona Preis	Consumer	United States	Westland	...	0.0	
45	Xylona Preis	Consumer	United States	Westland	...	0.0	

	Profit	7_day_avg_sales	Month	Cohort	Order_Month	Cohort_Month	\
0	5.5512	NaN	2019-01	2019-01	2019-01	2019-01	
1	6.8922	NaN	2019-01	2019-01	2019-01	2019-01	
2	4.2717	NaN	2019-01	2019-01	2019-01	2019-01	
3	6.8922	NaN	2019-01	2019-01	2019-01	2019-01	
14	8.0406	270.249714	2019-01	2019-01	2019-01	2019-01	
15	6.8922	184.599714	2019-01	2019-01	2019-01	2019-01	
16	1.1680	181.488857	2019-01	2019-01	2019-01	2019-01	
17	9.7500	185.011714	2019-01	2019-01	2019-01	2019-01	
44	4.8609	218.018286	2019-01	2019-01	2019-01	2019-01	
45	30.0234	247.743429	2019-01	2019-01	2019-01	2019-01	

	Sales_Rank	Sales_Zscore	Cumulative_Sales
0	1672.0	-0.341629	16.448
1	2214.0	-0.362451	19.988
2	1852.0	-0.349152	31.772
3	471.0	0.071801	304.508
14	941.0	-0.244388	381.236
15	1907.5	-0.351336	391.666
16	1957.5	-0.353088	401.010
17	1354.0	-0.317831	432.210
44	1749.0	-0.344400	446.940
45	469.5	0.072130	719.880

[10 rows x 29 columns]

In [19]:

```
# 10. Write a function to return total sales, average discount, and max profit per  
# category.
```

```
def category_summary(df):  
    summary = df.groupby('Category').agg(  
        Total_Sales=('Sales', 'sum'),  
        Avg_Discount=('Discount', 'mean'),  
        Max_Profit=('Profit', 'max')  
    ).reset_index()  
    return summary
```

```
print(category_summary(df))
```

```

stdout:
      Category  Total_Sales  Avg_Discount  Max_Profit
0      Furniture  754747.7613      0.172962    1013.127
1  Office Supplies  731893.3140      0.156348    4946.370
2      Technology  839893.2790      0.131475    8399.976

```

```
In [20]:
```

```
# 11. Write a function to extract the first word from Product Name.
```

```

def first_word_product(df):
    df['First_Word'] = df['Product Name'].str.split().str[0]
    return df

print(first_word_product(df).head())

```

```

stdout:
      Row ID      Order ID Order Date  Ship Date      Ship Mode Customer ID \
0         1  US-2019-103800  2019-01-03  07-01-2019  Standard Class  DP-13000
1         2  US-2019-112326  2019-01-04  08-01-2019  Standard Class  PO-19195
2         3  US-2019-112326  2019-01-04  08-01-2019  Standard Class  PO-19195
3         4  US-2019-112326  2019-01-04  08-01-2019  Standard Class  PO-19195
4         5  US-2019-141817  2019-01-05  12-01-2019  Standard Class  MB-18085

      Customer Name      Segment Country/Region      City ... Discount \
0  Darren Powers      Consumer  United States      Houston ...      0.2
1  Phillina Ober      Home Office  United States  Naperville ...      0.8
2  Phillina Ober      Home Office  United States  Naperville ...      0.2
3  Phillina Ober      Home Office  United States  Naperville ...      0.2
4    Mick Brown      Consumer  United States  Philadelphia ...      0.2

      Profit 7_day_avg_sales      Month      Cohort Order_Month Cohort_Month \
0    5.5512             NaN  2019-01  2019-01      2019-01      2019-01
1    6.8922             NaN  2019-01  2019-01      2019-01      2019-01
2    4.2717             NaN  2019-01  2019-01      2019-01      2019-01
3    6.8922             NaN  2019-01  2019-01      2019-01      2019-01
4    4.8840             NaN  2019-01  2019-01      2019-01      2019-01

      Sales_Rank  Sales_Zscore  First_Word
0         1672.0      -0.341629      Message
1         2214.0      -0.362451          GBC
2         1852.0      -0.349152          Avery
3          471.0       0.071801          SAFCO
4         2156.0      -0.336647          Avery

```

```
[5 rows x 29 columns]
```

```

In [24]:

# 12. Write a function to label orders as High, Medium, or Loss based on Profit.

import numpy as np

def label_profit_orders(df):
    conditions = [
        df['Profit'] > 100,          # High profit
        (df['Profit'] > 0) & (df['Profit'] <= 100), # Medium profit
        df['Profit'] <= 0           # Loss
    ]
    choices = ['High', 'Medium', 'Loss']
    df['Profit_Label'] = np.select(conditions, choices, default='Unknown')
    return df

# Example usage
print(label_profit_orders(df).head())

```

stdout:

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	\
0	1	US-2019-103800	2019-01-03	07-01-2019	Standard Class	DP-13000	
1	2	US-2019-112326	2019-01-04	08-01-2019	Standard Class	PO-19195	
2	3	US-2019-112326	2019-01-04	08-01-2019	Standard Class	PO-19195	
3	4	US-2019-112326	2019-01-04	08-01-2019	Standard Class	PO-19195	
4	5	US-2019-141817	2019-01-05	12-01-2019	Standard Class	MB-18085	

	Customer Name	Segment	Country/Region	City	...	Profit	\
0	Darren Powers	Consumer	United States	Houston	...	5.5512	
1	Phillina Ober	Home Office	United States	Naperville	...	6.8922	
2	Phillina Ober	Home Office	United States	Naperville	...	4.2717	
3	Phillina Ober	Home Office	United States	Naperville	...	6.8922	
4	Mick Brown	Consumer	United States	Philadelphia	...	4.8840	

	7_day_avg_sales	Month	Cohort	Order_Month	Cohort_Month	Sales_Rank	\
0	NaN	2019-01	2019-01	2019-01	2019-01	1672.0	
1	NaN	2019-01	2019-01	2019-01	2019-01	2214.0	
2	NaN	2019-01	2019-01	2019-01	2019-01	1852.0	
3	NaN	2019-01	2019-01	2019-01	2019-01	471.0	
4	NaN	2019-01	2019-01	2019-01	2019-01	2156.0	

	Sales_Zscore	First_Word	Profit_Label
0	-0.341629	Message	Medium
1	-0.362451	GBC	Medium
2	-0.349152	Avery	Medium
3	0.071801	SAFCO	Medium
4	-0.336647	Avery	Medium

[5 rows x 30 columns]

In [25]:

```
# 13. Write a function to create a pivot table with Region, Ship Mode, and mean  
# Profit.
```

```
def pivot_profit(df):  
    pivot = pd.pivot_table(  
        df,  
        values='Profit',  
        index=['Region', 'Ship Mode'],  
        columns='Category',  
        aggfunc='mean',  
        fill_value=0  
    )  
    return pivot
```

```
print(pivot_profit(df))
```

stdout:

Category		Furniture	Office Supplies	Technology
Central	First Class	30.097220	32.034373	60.707510
	Same Day	36.760825	17.087064	81.358961
	Second Class	39.981191	36.080358	57.878708
	Standard Class	42.264165	31.279474	113.046422
East	First Class	37.293477	32.049367	137.803973
	Same Day	45.550523	24.764594	203.789954
	Second Class	36.968306	31.039381	72.454941
	Standard Class	36.922249	30.285779	142.640950
South	First Class	56.552240	24.782781	77.285414
	Same Day	24.272662	26.588531	47.111844
	Second Class	61.466315	27.291624	166.601233
	Standard Class	48.925407	34.120594	84.084494
West	First Class	31.240935	28.870444	146.627825
	Same Day	29.622493	26.526086	148.453655
	Second Class	43.794648	31.428103	66.760846
	Standard Class	35.296539	31.141093	68.505651

In [28]:

```
# 14. Write a function to filter sales between two given dates.
```

```
def filter_sales_dates(df, start_date, end_date):  
    temp_df = df.copy() # avoid changing original df  
    temp_df['Order Date'] = pd.to_datetime(temp_df['Order Date'], dayfirst=True)  
    return temp_df[(temp_df['Order Date'] >= start_date) & (temp_df['Order Date'] <= end_date)]
```

```
print(filter_sales_dates(df, '2019-01-03', '2019-12-30'))
```

stdout:

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	\
0	1	US-2019-103800	2019-01-03	07-01-2019	Standard Class	
1	2	US-2019-112326	2019-01-04	08-01-2019	Standard Class	
2	3	US-2019-112326	2019-01-04	08-01-2019	Standard Class	
3	4	US-2019-112326	2019-01-04	08-01-2019	Standard Class	
4	5	US-2019-141817	2019-01-05	12-01-2019	Standard Class	
...	...	...	...	...	...	
2032	2033	US-2019-138072	2019-12-30	03-01-2020	Standard Class	
2033	2034	US-2019-104738	2019-12-30	01-01-2020	Second Class	
2034	2035	US-2019-115189	2019-12-30	03-01-2020	Second Class	
2035	2036	US-2019-104738	2019-12-30	01-01-2020	Second Class	
2036	2037	CA-2019-143500	2019-12-30	03-01-2020	Standard Class	

	Customer ID	Customer Name	Segment	Country/Region	City	\
0	DP-13000	Darren Powers	Consumer	United States	Houston	
1	PO-19195	Phillina Ober	Home Office	United States	Naperville	
2	PO-19195	Phillina Ober	Home Office	United States	Naperville	
3	PO-19195	Phillina Ober	Home Office	United States	Naperville	
4	MB-18085	Mick Brown	Consumer	United States	Philadelphia	
...	...	...	...	...	...	
2032	GA-14725	Guy Armstrong	Consumer	United States	Philadelphia	
2033	SP-20620	Stefania Perrino	Corporate	United States	Laredo	
2034	AR-10345	Alex Russell	Corporate	United States	Philadelphia	
2035	SP-20620	Stefania Perrino	Corporate	United States	Laredo	
2036	HO-15230	Harry Olson	Consumer	Canada	Charlottetown	

	...	Profit	7_day_avg_sales	Month	Cohort	Order_Month	Cohort_Month	\
0	...	5.5512	NaN	2019-01	2019-01	2019-01	2019-01	
1	...	6.8922	NaN	2019-01	2019-01	2019-01	2019-01	
2	...	4.2717	NaN	2019-01	2019-01	2019-01	2019-01	
3	...	6.8922	NaN	2019-01	2019-01	2019-01	2019-01	
4	...	4.8840	NaN	2019-01	2019-01	2019-01	2019-01	
...	...	...	...	...	...	...	...	
2032	...	6.9920	102.540429	2019-12	2019-02	2019-12	2019-02	
2033	...	28.7679	149.181857	2019-12	2019-09	2019-12	2019-09	
2034	...	25.0182	180.112714	2019-12	2019-09	2019-12	2019-09	
2035	...	19.0386	132.341143	2019-12	2019-09	2019-12	2019-09	
2036	...	6.8922	130.918286	2019-12	2019-12	2019-12	2019-12	

	Sales_Rank	Sales_Zscore	First_Word	Profit_Label
0	1672.0	-0.341629	Message	Medium
1	2214.0	-0.362451	GBC	Medium
2	1852.0	-0.349152	Avery	Medium
3	471.0	0.071801	SAFCO	Medium
4	2156.0	-0.336647	Avery	Medium
...	...	...	...	...
2032	1918.0	-0.323045	Maxell	Medium
2033	399.0	0.162202	AT&T	Medium
2034	651.0	0.038293	ClearSounds	Medium
2035	552.0	-0.017167	Plantronics	Medium
2036	2937.5	-0.363283	Wilson	Medium

[2037 rows x 30 columns]

```

In [41]:

# 15. Write a function to merge Orders and Returns data and identify returned
# orders.

returned_orders = ['US-2019-103800', 'US-2019-112326'] # example order IDs

def merge_orders_returns(df, returned_orders):
    df['Returned'] = df['Order ID'].isin(returned_orders)
    return df

merged_df = merge_orders_returns(df, returned_orders)
print(merged_df.head())

```

```

stdout:

   Row ID   Order ID Order Date   Ship Date   Ship Mode Customer ID \
0      1  US-2019-103800 2019-01-03  07-01-2019  Standard Class  DP-13000
1      2  US-2019-112326 2019-01-04  08-01-2019  Standard Class  PO-19195
2      3  US-2019-112326 2019-01-04  08-01-2019  Standard Class  PO-19195
3      4  US-2019-112326 2019-01-04  08-01-2019  Standard Class  PO-19195
4      5  US-2019-141817 2019-01-05  12-01-2019  Standard Class  MB-18085

   Customer Name   Segment Country/Region   City ... \
0  Darren Powers   Consumer   United States   Houston ...
1  Phillina Ober   Home Office   United States   Naperville ...
2  Phillina Ober   Home Office   United States   Naperville ...
3  Phillina Ober   Home Office   United States   Naperville ...
4    Mick Brown   Consumer   United States   Philadelphia ...

   7_day_avg_sales   Month   Cohort Order_Month Cohort_Month Sales_Rank \
0             NaN  2019-01  2019-01    2019-01    2019-01    1672.0
1             NaN  2019-01  2019-01    2019-01    2019-01    2214.0
2             NaN  2019-01  2019-01    2019-01    2019-01    1852.0
3             NaN  2019-01  2019-01    2019-01    2019-01     471.0
4             NaN  2019-01  2019-01    2019-01    2019-01    2156.0

   Sales_Zscore First_Word Profit_Label Returned
0    -0.341629    Message    Medium    True
1    -0.362451         GBC    Medium    True
2    -0.349152     Avery    Medium    True
3     0.071801     SAFCO    Medium    True
4    -0.336647     Avery    Medium   False

[5 rows x 31 columns]

```

```

In [42]:

# 16. Write a function to remove outliers using IQR method.

def remove_outliers_iqr(df, column='Sales'):
    # Calculate Q1, Q3 and IQR
    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)
    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    return df[(df[column] >= lower) & (df[column] <= upper)]

filtered_df = remove_outliers_iqr(df)
print(filtered_df.head())

```

stdout:

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	\
0	1	US-2019-103800	2019-01-03	07-01-2019	Standard Class	DP-13000	
1	2	US-2019-112326	2019-01-04	08-01-2019	Standard Class	PO-19195	
2	3	US-2019-112326	2019-01-04	08-01-2019	Standard Class	PO-19195	
3	4	US-2019-112326	2019-01-04	08-01-2019	Standard Class	PO-19195	
4	5	US-2019-141817	2019-01-05	12-01-2019	Standard Class	MB-18085	

	Customer Name	Segment	Country/Region	City	...	\
0	Darren Powers	Consumer	United States	Houston	...	
1	Phillina Ober	Home Office	United States	Naperville	...	
2	Phillina Ober	Home Office	United States	Naperville	...	
3	Phillina Ober	Home Office	United States	Naperville	...	
4	Mick Brown	Consumer	United States	Philadelphia	...	

	7_day_avg_sales	Month	Cohort	Order_Month	Cohort_Month	Sales_Rank	\
0	NaN	2019-01	2019-01	2019-01	2019-01	1672.0	
1	NaN	2019-01	2019-01	2019-01	2019-01	2214.0	
2	NaN	2019-01	2019-01	2019-01	2019-01	1852.0	
3	NaN	2019-01	2019-01	2019-01	2019-01	471.0	
4	NaN	2019-01	2019-01	2019-01	2019-01	2156.0	

	Sales_Zscore	First_Word	Profit_Label	Returned
0	-0.341629	Message	Medium	True
1	-0.362451	GBC	Medium	True
2	-0.349152	Avery	Medium	True
3	0.071801	SAFCO	Medium	True
4	-0.336647	Avery	Medium	False

[5 rows x 31 columns]

In [43]:

```
# 17. Write a function to return top 3 products by sales in each category
```

```
def top_products_by_category(df, top_n=3):
    df_sorted = df.sort_values(['Category', 'Sales'], ascending=[True, False])

    top_products = df_sorted.groupby('Category').head(top_n)

    return top_products

print(top_products_by_category(df, top_n=3))
```

stdout:

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	\
8347	8348	US-2022-118892	2022-08-17	22-08-2022	Second Class	
3584	3585	US-2020-117086	2020-11-08	12-11-2020	Standard Class	
2108	2109	US-2020-116638	2020-01-28	31-01-2020	Second Class	
6674	6675	US-2021-117121	2021-12-17	21-12-2021	Standard Class	
1203	1204	US-2019-116904	2019-09-23	28-09-2019	Standard Class	
811	812	US-2019-143917	2019-07-25	27-07-2019	Second Class	
226	227	US-2019-145317	2019-03-18	23-03-2019	Standard Class	
5894	5895	US-2021-118689	2021-10-02	09-10-2021	Standard Class	
7255	7256	US-2022-140151	2022-03-23	25-03-2022	First Class	

	Customer ID	Customer Name	Segment	Country/Region	City	\
8347	TP-21415	Tom Prescott	Consumer	United States	Philadelphia	
3584	QJ-19255	Quincy Jones	Corporate	United States	Burlington	
2108	JH-15985	Joseph Holt	Consumer	United States	Concord	
6674	AB-10105	Adrian Barton	Consumer	United States	Detroit	
1203	SC-20095	Sanjit Chand	Consumer	United States	Minneapolis	
811	KL-16645	Ken Lonsdale	Consumer	United States	San Francisco	
226	SM-20320	Sean Miller	Home Office	United States	Jacksonville	
5894	TC-20980	Tamara Chand	Corporate	United States	Lafayette	
7255	RB-19360	Raymond Buch	Consumer	United States	Seattle	

	...	7_day_avg_sales	Month	Cohort	Order_Month	Cohort_Month	\
8347	...	931.600286	2022-08	2020-11	2022-08	2020-11	
3584	...	723.623143	2020-11	2019-03	2020-11	2019-03	
2108	...	1152.390857	2020-01	2019-08	2020-01	2019-08	
6674	...	1789.257429	2021-12	2019-12	2021-12	2019-12	
1203	...	1492.913571	2019-09	2019-02	2019-09	2019-02	
811	...	1465.262000	2019-07	2019-07	2019-07	2019-07	
226	...	3704.852286	2019-03	2019-03	2019-03	2019-03	
5894	...	2636.138857	2021-10	2019-11	2021-10	2019-11	
7255	...	2086.318286	2022-03	2021-04	2022-03	2021-04	

	Sales_Rank	Sales_Zscore	First_Word	Profit_Label	Returned
8347	13.0	6.755770	HON	Medium	False
3584	14.0	6.737584	Riverside	High	False
2108	6.0	6.564564	Chromcraft	Medium	False
6674	2.0	15.590269	GBC	High	False
1203	3.0	14.875984	Ibico	High	False
811	2.0	12.839710	High	High	False
226	1.0	36.151003	Cisco	Medium	False
5894	1.0	27.861806	Canon	High	False
7255	1.0	22.215813	Canon	High	False

[9 rows x 31 columns]

In [46]:

```
# 18. Write a function to resample daily sales into monthly summary.

def monthly_sales_summary(df):
    df['Order Date'] = pd.to_datetime(df['Order Date'], dayfirst=True)

    df = df.set_index('Order Date')

    monthly_summary = df['Sales'].resample('ME').sum().reset_index()

    return monthly_summary

print(monthly_sales_summary(df))
```

stdout:

	Order Date	Sales
0	2019-01-31	14518.0550
1	2019-02-28	4519.8920
2	2019-03-31	56933.9090
3	2019-04-30	28295.3450
4	2019-05-31	26319.7670
5	2019-06-30	34669.4796
6	2019-07-31	33946.3930
7	2019-08-31	28918.3385
8	2019-09-30	82670.4288
9	2019-10-31	32413.3390
10	2019-11-30	78826.9567
11	2019-12-31	72008.3085
12	2020-01-31	18461.9156
13	2020-02-29	11951.4110
14	2020-03-31	39978.6280
15	2020-04-30	34195.2085
16	2020-05-31	30246.9465
17	2020-06-30	24797.2920
18	2020-07-31	28765.3250
19	2020-08-31	37057.0042
20	2020-09-30	64627.0460
21	2020-10-31	31407.1335
22	2020-11-30	75972.5635
23	2020-12-31	75532.5572
24	2021-01-31	18830.3310
25	2021-02-28	22978.8150
26	2021-03-31	53031.0690
27	2021-04-30	38829.1590
28	2021-05-31	57042.8380
29	2021-06-30	40937.1480
30	2021-07-31	40300.4990
31	2021-08-31	31716.8143
32	2021-09-30	73521.6629
33	2021-10-31	59831.0010
34	2021-11-30	79411.9658
35	2021-12-31	97502.2770
36	2022-01-31	44259.2140
37	2022-02-28	20301.1334
38	2022-03-31	60728.4808
39	2022-04-30	36779.0361
40	2022-05-31	45155.4822
41	2022-06-30	53056.0777
42	2022-07-31	45989.4960
43	2022-08-31	64129.7580
44	2022-09-30	88064.5320
45	2022-10-31	83474.7832
46	2022-11-30	118454.5050
47	2022-12-31	85175.0328

```

In [51]:

# 19. Write a function pipeline to filter Technology, remove outliers, and group by
# Sub-Category.

def tech_sales_pipeline(df):
    tech_df = df[df['Category'] == 'Technology']

    Q1 = tech_df['Sales'].quantile(0.25)
    Q3 = tech_df['Sales'].quantile(0.75)
    IQR = Q3 - Q1
    tech_df = tech_df[(tech_df['Sales'] >= Q1 - 1.5*IQR) & (tech_df['Sales'] <= Q3 + 1.5*IQR)]

    result = tech_df.groupby('Sub-Category')['Sales'].sum()

    return result

print(tech_sales_pipeline(df))

```

```

stdout:
Sub-Category
Accessories    121400.880
Copiers        21876.710
Machines       28910.820
Phones         219787.626
Name: Sales, dtype: float64

```

```

In [64]:

# 20. Rewrite an apply-based function using vectorized Pandas operations and
# compare performance.

import pandas as pd
import numpy as np
import time

df = pd.DataFrame({'Sales': np.random.randint(1, 1000, 1000000)})

# Apply-based
start = time.time()
df['Sales_sq_apply'] = df['Sales'].apply(lambda x: x**2)
end = time.time()
print("Apply time:", end - start)

# Vectorized
start = time.time()
df['Sales_sq_vec'] = df['Sales'] ** 2
end = time.time()
print("Vectorized time:", end - start)

```

```

stdout:
Apply time: 0.32200026512145996
Vectorized time: 0.0009999275207519531

```

```

..

```